

PROYECTO FINAL: AVANCE MISIÓN 1

1. Propósito del sistema

Se propone diseñar un sistema de gestión de pedidos web.

La plataforma facilita a los **usuarios** hacer seguimiento de sus pedidos realizados a un local en particular, evitando el contacto directo del **usuario** con servicio al cliente, mejorando así la experiencia de usuario.

La plataforma, a su vez, también facilita la gestión, control y seguimiento de todos los pedidos realizados por los **usuarios** haciendo más manejable la gestión administrativa por parte de los **clientes** del software.

Los **clientes** son las organizaciones con la necesidad de tener mayor observabilidad en el estado de los pedidos de sus **usuarios**.

Los **usuarios** son personas externas que hacen uso de los servicios de los **clientes**

Objetivos funcionales principales:

- El sistema tomará los pedidos de los **usuarios** y los guardará en base de datos
- El sistema mostrará el estado de los pedidos de los **usuarios**
- El sistema notificará a los **usuarios** y **clientes** por medio de email cualquier cambio en el estado de los pedidos

2. Alcance de la primera versión

- Funcionalidades incluidas:
 - Toma de pedidos
 - Seguimiento de pedidos
 - Notificación por email de cambio de estados en los pedidos
- Exclusiones
 - Pagos en plataforma
 - Seguimiento en tiempo real por mapas interactivos del domicilio de los pedidos
 - Cuentas de **usuario**

3. Identifica los contenedores o servicios

Contenedor	Rol/Función	Tecnología principal
Frontend Web	Interfaz de usuario	React
Backend	Servicio de pedidos	.NET
Backend	Servicio de administración	Kotlin
Backend	Servicio de notificaciones	Python

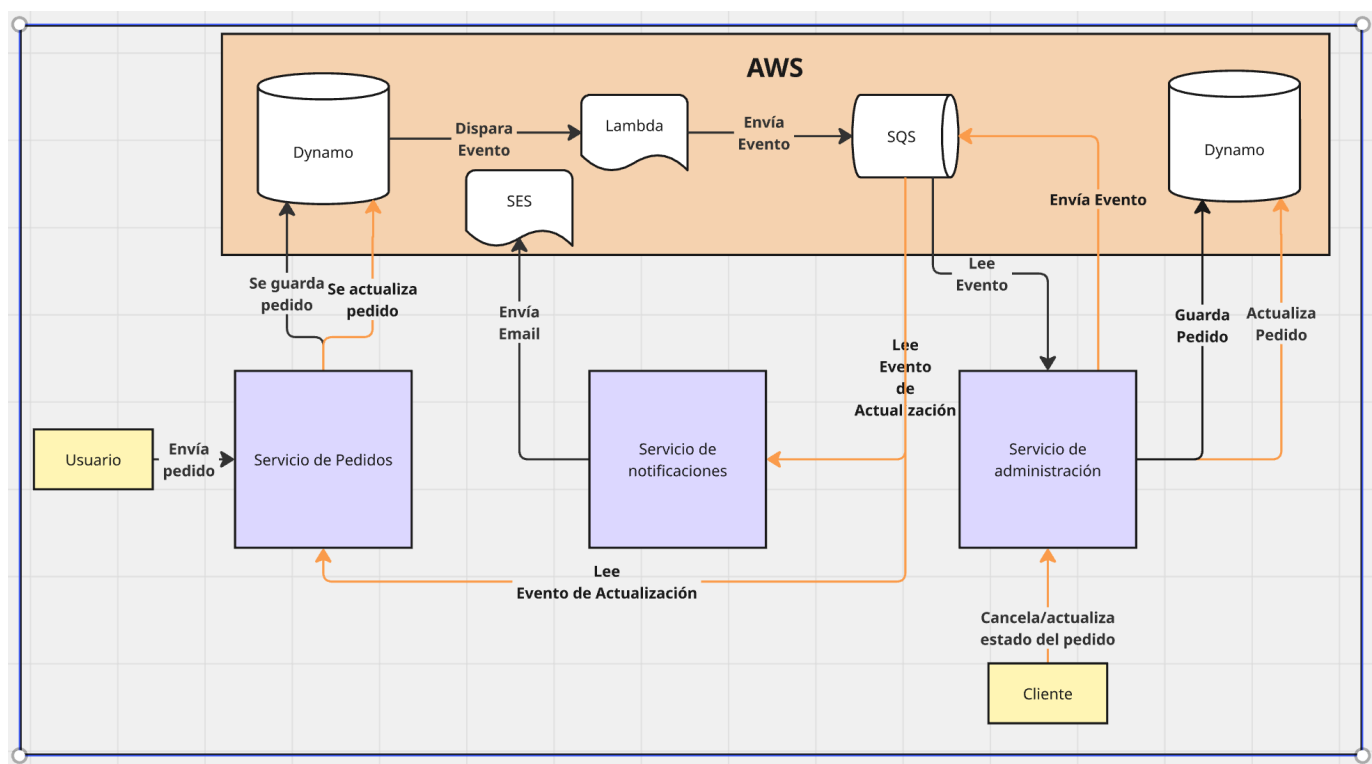
- **Servicio de pedidos:** será el encargado de tomar los pedidos de los **usuarios**, validar que toda la información ingresada sea válida y persistir esta información en base de datos.
- **Servicio de administración:** este servicio será usado por el cliente para hacer cambios de estado en los pedidos de los **usuarios** y a su vez generar nuevos eventos de actualización.
- **Servicio de notificaciones:** leerá eventos y notificará al **usuario** y al **cliente** de cualquier cambio en el estado de los pedidos

4. Define cómo se comunican los contenedores

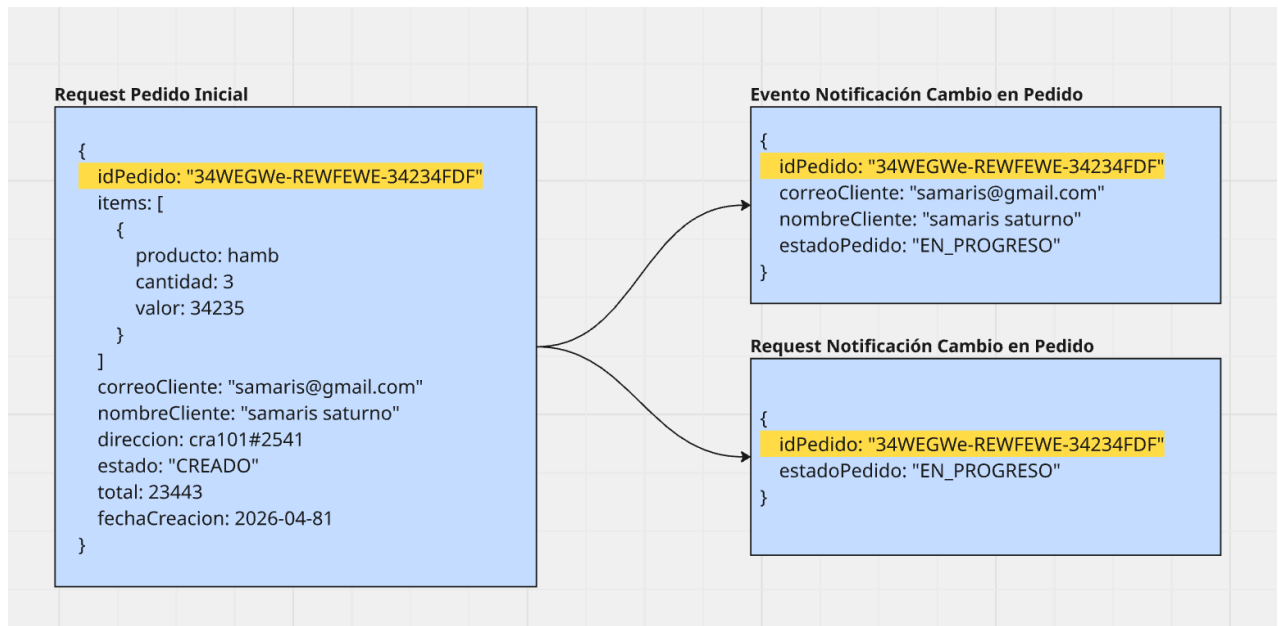
Proponemos una comunicación desacoplada a través de eventos usando los servicios de AWS Dynamo y Dynamo Streams, Lambda, SQS y SES. Los contenedores no tendrán comunicación directa, sino que cada uno trabajará a su propio ritmo según los eventos que se vayan generando en el sistema.

Entre el cliente y el servicio de pedidos proponemos una comunicación sincrónica HTTP mediante endpoints API Rest.

5. Diseña la arquitectura general



- **Modelo de datos NoSQL**



El modelo anterior representa todos los objetos nosql que serán utilizado a lo largo de toda nuestra aplicación, al ser un MVP los modelos de datos son sencillos y concisos, justo lo necesario para completar el proyecto y la idea principal de negocio

El *Request Pedido Inicial* será también la única unidad de información que almacenaremos en base de datos y de esta información se conformarán los demás objetos que se utilizarán en la aplicación.

Justificación tecnológica

React fue seleccionado para el frontend por ser la biblioteca de interfaces de usuario más adoptada de la industria. Su modelo basado en componentes reutilizables permite construir una interfaz modular para la visualización y seguimiento de pedidos, y su naturaleza SPA mejora la experiencia de usuario al evitar recargas completas de página.

.NET fue elegido para el servicio de pedidos por ser una plataforma robusta y de alto rendimiento, ampliamente probada en la construcción de APIs REST empresariales. Ofrece herramientas maduras para validación de datos y manejo de excepciones, y tiene soporte oficial para contenerización con Docker y despliegue en Kubernetes.

Kotlin fue seleccionado para el servicio de administración por su sintaxis concisa, su seguridad en tiempo de compilación y su madurez en entornos de microservicios. Spring Boot sobre Kotlin permite construir APIs REST de forma rápida y estructurada, y su soporte nativo para corrutinas lo hace adecuado para manejar operaciones concurrentes de actualización de pedidos.

Python fue elegido para el servicio de notificaciones por su simplicidad y su velocidad de integración con servicios externos. La librería boto3 simplifica enormemente la conexión con

SQS y SES, y su bajo tiempo de arranque y naturaleza ligera lo hacen ideal para un servicio con una responsabilidad acotada y orientado a eventos.

DynamoDB fue elegido como motor de persistencia por su modelo NoSQL orientado a documentos, que se adapta naturalmente a la estructura variable de un pedido. No requiere esquemas rígidos, escala automáticamente y replica los datos en múltiples zonas de disponibilidad, garantizando durabilidad sin administrar infraestructura de base de datos.

DynamoDB Streams actúa como el disparador de eventos del sistema. Captura en tiempo real cada inserción o modificación de un pedido y lo publica como evento, permitiendo que el resto de la arquitectura reaccione de forma desacoplada sin comunicación directa entre servicios.

AWS Lambda conecta DynamoDB Streams con SQS como procesador serverless de eventos. Se ejecuta únicamente cuando hay un evento que procesar, escala automáticamente según el volumen de cambios y elimina la necesidad de mantener un servidor dedicado para esta tarea.

Amazon SQS actúa como buffer entre Lambda y el servicio de notificaciones. Si el servicio de notificaciones cae temporalmente, los eventos permanecen encolados y se procesan al recuperarse, garantizando que ningún cambio de estado quede sin notificar y protegiendo el sistema ante picos de carga.

Amazon SES fue elegido para el envío de correos transaccionales por su alta tasa de entrega, su integración nativa con AWS y su bajo costo por mensaje. Al ser completamente administrado, elimina la necesidad de gestionar servidores de correo propios y se integra con Python en pocas líneas mediante boto3.

6. Selecciona los servicios de nube

La solución correrá en AWS con un enfoque principal en EKS (Amazon Elastic Kubernetes Service) para el entorno objetivo del proyecto. EKS nos permite orquestar los microservicios en .NET, Kotlin y Python de forma estandarizada, con alta disponibilidad, escalabilidad y operación centralizada.

Para desarrollo local, se puede usar Docker Compose o Minikube como entorno de pruebas previas, pero el despliegue oficial será sobre EKS.

Uso de servicios de nube: Se utilizarán servicios administrados de AWS para reducir complejidad operativa y mejorar confiabilidad:

- **EKS:** ejecución y orquestación de contenedores.
- **ECR:** almacenamiento de imágenes Docker de los microservicios.
- **DynamoDB:** persistencia de pedidos (fuente única de verdad).
- **DynamoDB Streams:** emisión de eventos por cambios de estado.
- **SQS:** mensajería desacoplada y procesamiento confiable.
- **SES:** envío de notificaciones por correo.
- **CloudWatch:** logs, métricas y alarmas. (no para esta fase, para mas adelante)

La solución, al estar compuesta por microservicios en .NET, Kotlin y Python, se desplegará en contenedores. El entorno objetivo será EKS en AWS, ya que permite administrar Kubernetes de manera gestionada, con mejor escalabilidad, resiliencia y gobierno operativo. Para desarrollo local (ambientes de pre-producción) se podrán usar Minikube, pero la arquitectura final se apoya en servicios cloud administrados como ECR, S3, DynamoDB, SQS y SES para lograr una plataforma desacoplada, observable y lista para crecimiento.

7. Define los volúmenes y persistencia

La solución maneja un volumen inicial bajo (50 pedidos/día, pico de 1 pedido/min), por lo que la persistencia se diseñó con servicios administrados y desacoplados. Los datos transaccionales de pedidos y estados se almacenan en DynamoDB, los cambios se propagan por DynamoDB Streams, y la entrega confiable de eventos se realiza con SQS. El frontend se almacena en S3 y la trazabilidad técnica en CloudWatch (misión 3). Los servicios en EKS se ejecutan como stateless, evitando dependencia de volúmenes locales o EFS para datos de negocio. Para este alcance no se requiere RDS.

- Volumen estimado: 50 pedidos por día.
- Pico estimado: 1 pedido por minuto.
- SLA de notificación: menor a 2 minutos.
- Retención funcional: hasta estado ENTREGADO + 60 días.

Datos persistentes

- **Datos del pedido:** idPedido, items, cliente, dirección, total, estado, fecha de creación, fecha de actualización, fecha de entrega.
- **Historial de estado del pedido:** al menos estado anterior, estado nuevo, timestamp y origen del cambio para auditoría básica.
- **Eventos de integración:** mensajes de cambio de estado mientras son procesados por el flujo de notificaciones.
- **Artefactos estáticos:** frontend web compilado.
- **Trazabilidad operativa:** logs y métricas para monitoreo y soporte.

Almacenamiento

- **DynamoDB:** almacena los pedidos como fuente principal de verdad.
- **DynamoDB Streams:** captura cambios de INSERT y MODIFY para disparar eventos.
- **SQS:** almacenamiento temporal y durable de mensajes de notificación hasta su consumo.
- **S3:** almacenamiento de frontend estático y opcionalmente reportes/exportaciones.
- **CloudWatch Logs:** persistencia de logs técnicos de microservicios y funciones.
- **ECR:** almacenamiento de imágenes de contenedores.
- **EKS:** microservicios stateless, sin persistencia crítica en disco local del pod.

8. Configura la seguridad básica

La seguridad de la plataforma se aplica en todas las capas. A nivel de código, se prohíbe el uso de credenciales en el repositorio y se integra SonarQube en el pipeline para análisis estático y detección de vulnerabilidades antes de cada despliegue. Los secretos (credenciales, claves API, tokens) se almacenan en AWS Secrets Manager y son consumidos en tiempo de ejecución por los pods mediante el Secrets Store CSI Driver. Los permisos entre servicios se gestionan con IRSA (IAM Roles for Service Accounts), garantizando mínimo privilegio por microservicio sin credenciales explícitas. A nivel de infraestructura, la arquitectura usa una VPC con subnets privadas para los pods y subnets públicas únicamente para el ALB, con VPC Endpoints para que el tráfico a DynamoDB, y SQS nunca salga a internet. DynamoDB y SQS están cifrados en reposo con KMS. El tráfico externo está protegido por WAF con reglas de SQLi, XSS y rate limiting. Toda la actividad es auditable en CloudTrail.

1. **Seguridad a nivel de código:** No hardcoding. Cero credenciales en el código fuente ni en archivos de configuración del repositorio.

Agregar .env y archivos de secrets al .gitignore desde el inicio del proyecto.

2. **Manejo de secretos con AWS Secrets Manager:** Todos los secretos viven en AWS Secrets Manager, nunca en el código.

Los pods en EKS los consumen en tiempo de ejecución mediante el Secrets Store CSI Driver (integración nativa EKS + Secrets Manager).

Ningún servicio conoce el valor de un secreto al momento del build, solo al momento de ejecución.

3. **SonarQube para análisis estático:** Integrar SonarQube en el pipeline CI/CD, el pipeline bloquea el despliegue si SonarQube detecta:
 - Vulnerabilidades de severidad alta o crítica.
 - Code smells que impliquen riesgos de seguridad (ej. SQL injection, hardcoded secrets).
 - Cobertura de pruebas por debajo del umbral definido.
 - SonarQube corre en cada Pull Request, no solo en main.

Secreto	Servicio que lo usa	Almacenado en
Credenciales de conexión a DynamoDB	Pedidos, Administración	IRSA - IAM Roles for Service Accounts
Clave de acceso a SQS	Notificaciones	IRSA - IAM Roles for Service Accounts
Credenciales SES (si se usa SMTP)	Notificaciones	Secrets Manager

Claves KMS para cifrado	Todos	Gestionado por AWS KMS, no expuesto
Tokens de Cognito (si se habilita)	Frontend, ALB	Secrets Manager / Variables de entorno seguras
API Keys de integraciones externas	Cualquier servicio	Secrets Manager

4. **Seguridad a nivel de infraestructura:** Redes privadas y subnets separadas; Los pods de EKS nunca tienen IP pública. Solo el ALB está en la subnet pública.

- **VPC Endpoints:** DynamoDB, S3, SQS y Secrets Manager accedidos por VPC Endpoints privados.
El tráfico entre EKS y estos servicios nunca sale a internet
- **WAF (Web Application Firewall):** WAF en ALB.

5. **Base de datos cifrada:** DynamoDB cifrado en reposo con AWS KMS (habilitado por defecto, pero recomendamos usar CMK — Customer Managed Key — para control total).

- SQS cifrado en reposo con KMS.
- Tráfico en tránsito cifrado con TLS en todos los endpoints de AWS (DynamoDB, SQS, SES).
- Frontend solo accesible por HTTPS

6. **Mínimo privilegio — Service Principal / IAM:** Cada componente tiene exactamente los permisos que necesita:

9. Identifica los criterios de éxito

- Un pedido creado debe verse reflejado al consumir el endpoint GET para obtener información de los pedidos.
- Al cambiar un estado de un pedido, la notificación de este evento debe llegar al correo electrónico del usuario.
- Si el servicio de notificaciones cae, las notificaciones deben quedar encoladas en el SQS esperando a que el servicio vuelva a estar operativo
- Tanto Clientes como Usuarios deben poder cancelar un pedido y el cliente debe poder actualizar el estado de los pedidos
- Cada servicio debe tener 2 réplicas para balanceo de carga

10. Prepara una presentación donde plantees la solución a los enunciados anteriores

AVANCE MISIÓN 1

Contenedores (Docker)

1. ¿Qué componentes de tu solución se beneficiarán de estar empaquetados como contenedores?

- Frontend web (React)
- Servicio de pedidos (.NET)
- Servicio de administración (Kotlin).
- Servicio de notificaciones (Python)

Estos se desplegarán como contenedores para garantizar la portabilidad, compatibilidad en SO y consistencia entre entornos.

2. ¿Vas a construir tus propias imágenes desde Dockerfiles o usarás imágenes oficiales de terceros?

Dockerizaremos cada componente con imágenes propias, por medio de Dockerfile, lo que permite tener mayor control sobre las dependencias, configuraciones y versiones de cada uno.

Frontend (React): Dockerfile multistage usando node:alpine para el build y nginx:alpine para servir el estático en runtime.

Servicio de pedidos (.NET): Dockerfile sobre mcr.microsoft.com/dotnet/aspnet.

Servicio de administración (Kotlin): Dockerfile sobre eclipse-temurin:21-jre-alpine.

Servicio de notificaciones (Python): Dockerfile sobre python:3.12-slim.

Esto facilita:

- Personalizar el entorno de ejecución según las necesidades del proyecto.
- Mantener consistencia entre desarrollo, pruebas y producción.
- Optimizar el tamaño y seguridad de las imágenes.
- Controlar versiones y despliegues de manera más precisa.

3. ¿Planeas ejecutar todos los contenedores desde el inicio, o habrá componentes opcionales según el entorno (desarrollo, producción)?

Ejecutaremos los componentes desde el inicio, en los entornos de desarrollo y producción, con el fin de mantener consistencia entre ambientes y garantizar que el comportamiento del sistema sea el mismo durante las pruebas y el despliegue final.

Aunque la configuración base será la misma, algunos parámetros como variables de entorno, recursos asignados o configuraciones específicas podrán variar según el entorno.

4. ¿Qué estrategia seguirás para el versionamiento de las imágenes?

Cada imagen generada seguirá Semantic Versioning (SemVer) con el formato MAJOR.MINOR.PATCH, donde:

- **MAJOR** incrementa con cambios incompatibles o reestructuraciones significativas del servicio.
- **MINOR** incrementa con nuevas funcionalidades compatibles con versiones anteriores.
- **PATCH** incrementa con correcciones de errores o ajustes menores.

Todas las imágenes se almacenan en Amazon ECR, con un repositorio por servicio. El tag se genera automáticamente desde el pipeline CI/CD al momento del build, tomando la versión del código fuente como fuente de verdad. Nunca se sobrescribe una imagen ya publicada con el mismo tag de versión, garantizando trazabilidad y la posibilidad de hacer rollback a cualquier versión anterior.

Orquestación (Kubernetes)

5. ¿Planeas ejecutar tu aplicación completa en Kubernetes o solo ciertos servicios?

Usaremos minikube para desarrollo local, desplegando los diferentes componentes (Front, Backend, BD) y en ambiente producción en Kubernetes EKS

Para simular DynamoDB en local se usará amazon/dynamodb-local como contenedor fuera del clúster.

6. ¿Qué ventajas te ofrece Kubernetes frente a Docker Compose para tu proyecto?

Kubernetes ofrece varias ventajas frente a Docker Compose, especialmente cuando el proyecto necesita escalabilidad, alta disponibilidad y manejo en producción.

Algunas ventajas importantes son:

- **Escalabilidad automática:** Kubernetes permite aumentar o disminuir instancias automáticamente según la carga del sistema. Docker Compose está más orientado a ambientes simples o de desarrollo.
- **Alta disponibilidad:** Si un contenedor falla, Kubernetes lo reinicia automáticamente o lo reemplaza en otro nodo, garantizando continuidad del servicio.
- **Orquestación avanzada:** Kubernetes administra despliegues, balanceo de carga, networking y comunicación entre microservicios de forma más robusta.
- **Despliegues sin downtime:** Permite realizar actualizaciones graduales (rolling updates) y rollback en caso de errores, evitando interrupciones del sistema.

- **Mejor manejo de microservicios:** Cuando el proyecto crece y tiene múltiples servicios, Kubernetes facilita la administración centralizada y organizada.
- **Gestión de recursos:** Se pueden definir límites de CPU y memoria para cada contenedor, optimizando el uso de infraestructura.
- **Portabilidad cloud:** Kubernetes funciona fácilmente en proveedores como Microsoft, Amazon o Google, facilitando migraciones y despliegues híbridos.
- **Mayor seguridad y configuración:** Permite manejar secretos, configuraciones, políticas de acceso y namespaces de forma más profesional.

7. ¿Cómo manejarás los despliegues de tus pods (usando Deployment, StatefulSet, etc.)?

Para el manejo de los despliegues se utilizará Deployment, ya que permite administrar aplicaciones stateless de forma sencilla y escalable.

Con Deployment se obtienen ventajas como:

- Actualizaciones automáticas y controladas mediante rolling updates.
- Rollback a versiones anteriores en caso de fallos.
- Escalabilidad aumentando o reduciendo réplicas de pods fácilmente.
- Autorecuperación, ya que Kubernetes recrea los pods si alguno falla.
- Alta disponibilidad distribuyendo varias réplicas del servicio.

StatefulSet no aplica para este proyecto porque ningún servicio requiere identidad de red persistente ni almacenamiento local entre reinicios. La persistencia es responsabilidad exclusiva de DynamoDB, que vive fuera del clúster.

8. ¿Necesitarás namespaces para separar entornos o componentes?

Sí, se creará un namespace por cada entorno (Dev-Prod), la finalidad es separar y aislar los recursos dentro del clúster de Kubernetes.

AWS EKS

9. ¿Tu solución final será desplegada en un clúster de EKS? ¿Por qué?

Sí. EKS es la opción para este proyecto porque permite orquestar los tres microservicios backend y uno frontend con alta disponibilidad, rolling updates y autorecuperación sin gestionar el control plane de Kubernetes manualmente. Al ser un servicio administrado, AWS se encarga de la disponibilidad del control plane, los parches de seguridad y la integración nativa con los demás servicios del ecosistema reduciendo así la carga operativa del equipo y permite enfocarnos en el producto.

10. ¿Usarás nodos EC2 o Fargate para ejecutar tus pods? ¿Qué criterios consideraste para elegir?

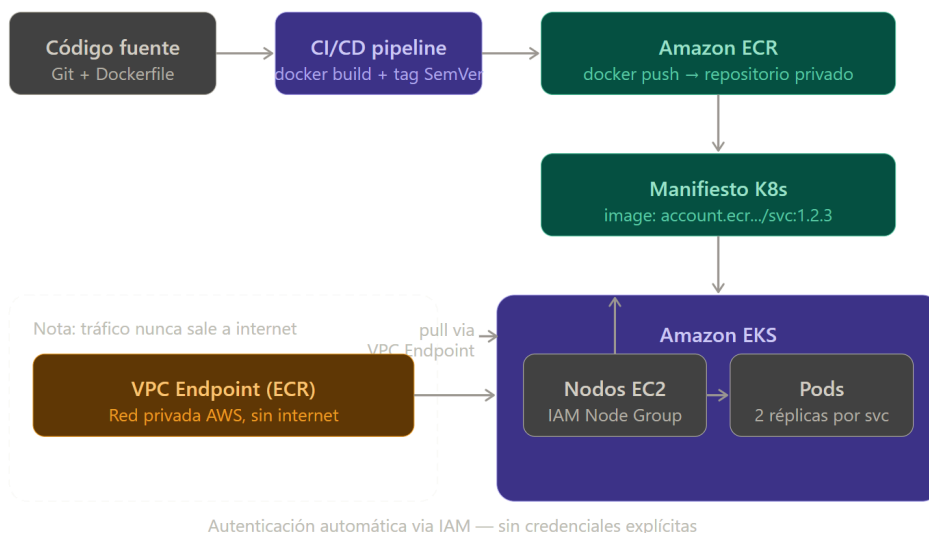
Se usarán nodos EC2 mediante un Node Group administrado. Los criterios fueron:

- **Costo:** EC2 es significativamente más económico que Fargate para cargas de trabajo continuas. Fargate cobra por vCPU y memoria por segundo, lo que resulta costoso cuando los pods corren las 24 horas.
- **Control:** EC2 permite elegir el tipo de instancia según el perfil de cada servicio y ajustar los recursos del nodo con más granularidad.
- **Volumen del proyecto:** Con solo tres servicios y un volumen inicial de 50 pedidos/día, el overhead de administrar un Node Group es mínimo y el ahorro en costo es relevante.
- **Compatibilidad:** IRSA, y el AWS Load Balancer Controller funcionan en ambas modalidades, por lo que EC2 no impone restricciones técnicas para este proyecto.

11. ¿Cómo subirás tus imágenes al entorno EKS? ¿Utilizarás Amazon ECR?

Sí. El flujo sería el siguiente:

- El pipeline CI/CD construye la imagen Docker y le asigna el tag SemVer correspondiente.
- La imagen se publica en Amazon ECR, en el repositorio del servicio correspondiente.
- El manifiesto de Kubernetes referencia la imagen por su URI completa en ECR (<account>.dkr.ecr.<region>.amazonaws.com/<servicio>:<version>).
- Al momento del despliegue, los nodos EC2 del clúster EKS descargan la imagen desde ECR usando los permisos IAM del Node Group, sin necesidad de credenciales explícitas.



12. ¿Cuál es el costo estimado del entorno si decides usar EKS en producción?

Estimación mensual — EKS producción · us-east-1 · 2× t3.small · sin S3 + CloudFront		
Componente	Detalle	Costo/mes
EKS control plane	1 clúster \$0.10/hr × 730 hrs	\$73.00
EC2 node group	2× t3.small \$0.0208/hr × 730 hrs × 2 nodos	\$30.37
ALB	1 balanceador \$0.0225/hr × 730 hrs (tarifa base)	~\$16.43
Amazon ECR	4 repositorios ~1 GB almacenado	~\$0.10
DynamoDB	50 pedidos/día, on-demand Lecturas y escrituras mínimas	~\$1–2
Amazon SQS	<1M mensajes/mes Cubierto por free tier	\$0.00
Amazon SES	~1 500 emails/mes \$0.10 por cada 1 000 emails	~\$0.15
Total estimado / mes		~\$121–123

Escalabilidad

13. ¿Tu solución necesita escalar horizontalmente (más réplicas)? ¿Cuáles componentes?

Sí, los tres servicios backend son candidatos a escalar horizontalmente porque son stateless y están detrás de un load balancer. La prioridad de escalado por componente es:

- **Servicio de pedidos:** el de mayor carga, ya que recibe todas las solicitudes entrantes de usuarios. Es el primero en necesitar réplicas adicionales ante un pico.
- **Servicio de administración:** carga moderada, usada principalmente por los clientes del software para gestionar estados.
- **Servicio de notificaciones:** escala en función del volumen de mensajes en SQS, no de tráfico HTTP directo.

14. ¿Qué métricas usarás como base para escalar (uso de CPU, memoria, número de conexiones, etc.)?

Se configurará un **Horizontal Pod Autoscaler (HPA)** en EKS con las siguientes métricas por servicio:

- **Servicio de pedidos:** CPU como métrica primaria (umbral del 70%). Un spike de CPU indica un aumento en el volumen de requests entrantes.
- **Servicio de administración:** CPU al 70%. La carga es predecible y correlaciona bien con uso de procesador.
- **Servicio de notificaciones:** longitud de la cola SQS como métrica externa. Si hay más de N mensajes pendientes, se agregan réplicas para consumirlos más rápido. Esto requiere el **KEDA (Kubernetes Event-Driven Autoscaling)**, que se integra nativamente con SQS.

15. ¿Vas a implementar un HPA (Horizontal Pod Autoscaler) para algún servicio?

Consideramos que el servicio de mayor demanda que podría ser escalado horizontalmente sería nuestro servicio de pedidos ya que, este servicio ofrece la posibilidad a los usuarios de consultar el estado de su pedido y sabemos que esta operación puede ser fácilmente explotada debido a que varias veces en el día, uno o varios clientes, pueden consultar esta información en repetidas ocasiones durante el día, o inclusive en minutos. Por eso consideramos que un escalado horizontal es una decisión acertada solo en este servicio.

16. ¿Tienes pensado escalar verticalmente algún recurso (por ejemplo, la base de datos RDS)?

Consideramos que la base de datos dynamo sería el único servicio que, dado un consumo masivo de información, debemos escalar tanto horizontal como verticalmente.

Resiliencia

17. ¿Qué mecanismos usarás para que tu aplicación se recupere de fallos?

- **Redundancia**
 - Tendremos réplicas de nuestros servicios y bases de datos en diferentes regiones de AWS para garantizar que ante cualquier falla nuestro sistema estará prestando servicio
- **Graceful degradation**
 - Al utilizar una arquitectura orientada a eventos estamos implícitamente utilizando la estrategia de resiliencia Graceful degradation ya que si, por ejemplo, alguno de nuestros sistemas está caído y no puede consumir los nuevos eventos, estos se irán encolando en nuestras colas de mensajería a la espera de que el servicio vuelva a levantarse y a consumirse. De esta manera las notificaciones a nuestros clientes pueden no llegar inmediatamente, pero sí llegarán en el transcurso del día.
- **Replicación**
 - Configuraremos réplicas de la base de datos para estar preparados en cualquier caso de pérdida de información

18. ¿Vas a configurar probes (liveness, readiness, startup) en tus contenedores?

Usaremos Startup, Readiness y Liveness probe para verificar constantemente si nuestros servicios están arriba, corriendo correctamente y recibiendo tráfico.

19. ¿Cómo protegerás los datos críticos ante fallos (respaldo de base de datos, réplicas, etc.)?

Realizaremos constantes réplicas de datos de la base de datos para estar seguros en este caso.

20. ¿Qué estrategia de actualización aplicarás a los pods: RollingUpdate o Recreate?

La estrategia que más se acomoda a nuestra arquitectura es RollingUpdate, ya que la naturaleza desacoplada y noSQL de nuestra aplicación nos permite recibir tráfico con peticiones de información diferentes y aun así seguir operando con naturalidad.

Consideraciones generales

21. ¿Cuáles decisiones de esta fase afectan directamente la forma en que avanzarás en las siguientes misiones?

- EKS como entorno de despliegue implicará crear los manifiestos de Kubernetes (Deployments, Services, Ingress, ConfigMaps, Secrets), lo que representa una carga de configuración significativa desde el inicio.
- IRSA y Secrets Manager como estrategia de seguridad nos exigirá configuración específica por cada pod, lo que debe tenerse presente al escribir los manifiestos
- La decisión de no tener cuentas de usuario simplifica el MVP pero obliga a que el seguimiento de pedidos se haga por ID o correo, lo cual debe reflejarse consistentemente en los endpoints y en la UI desde el principio
- La arquitectura orientada a eventos hace que cualquier cambio en el formato de eventos o en los estados del pedido afectará al menos tres componentes a la vez.

22. ¿Qué conceptos decidiste no aplicar (por ahora)? ¿Por qué?

- **Pagos en plataforma:** requiere integración con pasarelas externas, lo que introduce complejidad que está fuera del alcance del MVP
- **Seguimientos en tiempo real por mapas:** implica integración con servicios de geolocalización y websockets, tecnologías que aumentan la complejidad sin aportar valor en esta fase.
- **Cuentas de usuario:** se excluyeron para reducir el tiempo de desarrollo, pero es una deuda técnica clara ya que hoy no hay forma de verificar la identidad de quien consulta un pedido.

23. ¿Cuál sería la dificultad principal de implementar tu proyecto completamente en EKS hoy mismo?

La dificultad principal no es técnica sino operativa y de curva de aprendizaje.

- **Kubernetes tiene una curva de aprendizaje pronunciada.** Desplegar en EKS no es solo escribir un Dockerfile — requiere dominar conceptos como Deployments, Services, ConfigMaps, Secrets, Ingress, HPA, y RBAC. Un equipo que no ha trabajado antes con Kubernetes puede invertir más tiempo aprendiendo la plataforma que construyendo el producto.
- **La configuración inicial de EKS es extensa.** Antes de desplegar el primer pod hay que provisionar el clúster, configurar el Node Group, instalar el AWS Load Balancer

Controller, el Secrets Store CSI Driver para Secrets Manager, configurar IRSA para cada servicio, crear los VPC Endpoints, definir los security groups, y estructurar el pipeline CI/CD con ECR. Todo esto es trabajo de infraestructura que no aporta valor de negocio directo, pero es prerequisite para que cualquier servicio funcione.

- **La integración entre servicios AWS requiere conocimiento específico.** Conectar correctamente IRSA con los permisos de DynamoDB y SQS, configurar KEDA para el autoscaling del servicio de notificaciones basado en la longitud de la cola, o asegurarse de que los VPC Endpoints estén bien configurados para que el tráfico nunca salga a internet — son detalles que no son obvios y que generan horas de debugging si no se tienen claros desde el inicio.

24. ¿Qué harías distinto si tu proyecto fuera a escalar de 10 usuarios a 10,000?

- **CQRS y separación de lectura/escritura:** con miles de usuarios consultando el estado de los pedidos simultáneamente, separar el modelo de escritura del de lectura reduciría la latencia y el costo
- **Caché:** Introducir Redis para cachear consultas frecuentes de estado de pedidos, evitando golpear dynamoDB en cada request.
- **Costos:** a ese volumen los servicios como DynamoDB on-demand podrían ser más caros que una capacidad aprovisionada bien calibrada.
- **Sharding de datos:** particionamiento horizontal que divide una base de datos grande en partes más pequeñas y manejables